

```

module ultrasonic(
input clk,
input start,
input echo,
output trigger,
output reg new_measure = 0,
output reg timeout = 0,
output reg [20:0] distance_raw = 0
);
parameter CLK_MHZ = 50;
parameter TRIGGER_PULSE_US = 12;
parameter TIMEOUT_MS = 25;
localparam COUNT_TRIGGER_PULSE = CLK_MHZ * TRIGGER_PULSE_US;
localparam COUNT_TIMEOUT = CLK_MHZ * TIMEOUT_MS * 1000;
reg [20:0] counter = 0;
reg [2:0] state = 0;
localparam IDLE=0, TRIG=1, WAIT_ECHO=2, MEASURE=3, DONE=4;
assign trigger = (state == TRIG);
always @(posedge clk) begin
case(state)
IDLE: begin
new_measure <= 0;
timeout <= 0;
if(start) begin
state <= TRIG;
counter <= 0;
end
end
TRIG: begin
if(counter >= COUNT_TRIGGER_PULSE) begin
state <= WAIT_ECHO;

```

```

counter <= 0;
end else counter <= counter + 1;
end
// FIXED WAIT_ECHO TIMEOUT
WAIT_ECHO: begin
if(echo) begin
state <= MEASURE;
counter <= 0;
end
else if(counter >= COUNT_TIMEOUT) begin
timeout <= 1;
state <= DONE;
end
else counter <= counter + 1;
end
MEASURE: begin
if(counter >= COUNT_TIMEOUT) begin
timeout <= 1;
state <= DONE;
end
else if(!echo) begin
distance_raw <= counter;
state <= DONE;
end
else counter <= counter + 1;
end
DONE: begin
new_measure <= 1;
state <= IDLE;
end
endcase

```

```

end

endmodule

// ===== USS SIMPLE =====

module uss_simple(
input clk,
input echo,
input start,
output trig,
output reg near = 0
);
parameter THRESHOLD_CM = 40;
localparam D = 2900;
wire new_measure;
wire [20:0] distance_raw;
ultrasonic U (
.clk(clk),
.start(start),
.echo(echo),
.trigger(trig),
.new_measure(new_measure),
.timeout(),
.distance_raw(distance_raw)
);
// FAST RESPONSE (no filter)
always @(posedge clk) begin
if(new_measure) begin
near <= (distance_raw < (THRESHOLD_CM * D));
end
end
endmodule

// ===== MOTOR DRIVER =====

```

```

module dc2(input clk,
input [2:0] dir,
output reg IN1L, IN2L, IN3R, IN4R, ENA, ENB
);
always @(posedge clk) begin
IN1L=0; IN2L=0; IN3R=0; IN4R=0; ENA=0; ENB=0;
case(dir)
3'b000: begin // STOP
IN1L=0; IN2L=0;
IN3R=0; IN4R=0;
ENA=0; ENB=0;
end
3'b001: begin // FORWARD
IN1L=1; IN2L=0;
IN3R=1; IN4R=0;
ENA=1; ENB=1;
end
3'b010: begin // BACKWARD
IN1L=0; IN2L=1;
IN3R=0; IN4R=1;
ENA=1; ENB=1;
end
3'b011: begin // LEFT
IN1L=0; IN2L=1;
IN3R=1; IN4R=0;
ENA=1; ENB=1;
end
3'b100: begin // RIGHT
IN1L=1; IN2L=0;
IN3R=0; IN4R=1;
ENA=1; ENB=1;

```

```

end
endcase
end
endmodule

// ===== TOP MODULE =====

module bot25(
input clk,
input echoF, echoB, echoL, echoR,
output trigF, trigB, trigL, trigR,
output IN1L, IN2L, IN3R, IN4R, ENA, ENB,
output ledF, ledB, ledL, ledR
);
// ----- SENSOR SCAN (5 ms) -----
reg [1:0] sensor_sel = 0;
reg [19:0] delay = 0;
always @(posedge clk) begin
if(delay == 20'd250_000) begin
delay <= 0;
sensor_sel <= sensor_sel + 1;
end else delay <= delay + 1;
end
reg start = 0;
always @(posedge clk) begin
start <= (delay == 0);
end
// ----- SENSOR INSTANCES -----
wire nearF, nearB, nearL, nearR;
wire startF = (sensor_sel==2'd0) ? start : 0;
wire startL = (sensor_sel==2'd1) ? start : 0;
wire startR = (sensor_sel==2'd2) ? start : 0;
wire startB = (sensor_sel==2'd3) ? start : 0;

```

```

uss_simple uF(.clk(clk), .echo(echoF), .start(startF), .trig(trigF), .near(nearF));
uss_simple uL(.clk(clk), .echo(echoL), .start(startL), .trig(trigL), .near(nearL));
uss_simple uR(.clk(clk), .echo(echoR), .start(startR), .trig(trigR), .near(nearR));
uss_simple uB(.clk(clk), .echo(echoB), .start(startB), .trig(trigB), .near(nearB));

// ----- LATCH SENSOR VALUES -----
reg nF=0, nL=0, nR=0, nB=0;

always @(posedge clk) begin

nF <= nearF;

nL <= nearL;

nR <= nearR;

nB <= nearB;

end

//

assign ledF = nF;

assign ledB = nB;

assign ledL = nL;

assign ledR = nR;

// ----- TURN HOLD LOGIC -----

reg [22:0] turn_timer = 0;

reg turning = 0;

reg [2:0] dir;

always @(posedge clk) begin

if(turning) begin

if(turn_timer > 0)

turn_timer <= turn_timer - 1;

else

turning <= 0;

end

else begin

if (!nF)

dir <= 3'b001; // FORWARD

```

```

else if (nF && !nL) begin
    dir <= 3'b100; // LEFT
    turning <= 1;
    turn_timer <= 23'd5_000_000; // ~100 ms
end

else if (nF && nL && !nR) begin
    dir <= 3'b011; // RIGHT
    turning <= 1;
    turn_timer <= 23'd5_000_000;
end

else if (nF && nL && nR && !nB)
    dir <= 3'b010; // BACKWARD
else
    dir <= 3'b000; // STOP
end
end

// ----- MOTOR DRIVER -----
dc2 m(.clk(clk),
    .dir(dir),
    .IN1L(IN1L), .IN2L(IN2L),
    .IN3R(IN3R), .IN4R(IN4R),
    .ENA(ENA), .ENB(ENB)
);
endmodule

```